

Escola Superior de Tecnologia e Gestão de Águeda



Sistemas Operativos

Técnico Superior Profissional em Redes e Sistemas Informáticos

Sincronização de Processos

Professor: Ivan Miguel Pires

2025/2026

OBJETIVOS DE APRENDIZAGEM

- Compreender o problema da secção crítica
- Conhecer race conditions e suas consequências
- Dominar exclusão mútua
- Explorar primitivas de sincronização:
 - Mutex locks
 - Semáforos
- Resolver problemas clássicos:
 - Produtor-Consumidor
 - Leitores-Escritores
- Compreender deadlock e starvation
- Conhecer avanços modernos (2024-2026):
 - Lock-free e wait-free algorithms
 - Transactional memory
 - Sincronização em many-core e GPU

PROBLEMA PRODUTOR-CONSUMIDOR

- Cenário clássico:
 - Produtor gera dados
 - Consumidor consome dados
 - Buffer partilhado
- Exemplos práticos:
 - Servidor web: produz páginas, cliente consome
 - Pipeline de dados
 - Print spooler
- Desafios:
 - Produtor não deve sobrescrever buffer cheio
 - Consumidor não deve ler buffer vazio
 - Sincronização necessária
- Variáveis partilhadas:
 - `buffer[BUFFER_SIZE]`
 - `in, out` (índices)
 - `count` (itens no buffer)



BUFFER PARTILHADO

```
#define BUFFER_SIZE 10

typedef struct {
    ...
} item;

item buffer[BUFFER_SIZE];
int in=0;
int out=0;
```

Buffer
(ou qualquer outra coisa partilhada)

PROCESSO PRODUTOR



Produz

Buffer
(ou qualquer outra coisa partilhada)

```
while (true) {  
    /* produce an item and put in nextProduced */  
    while (count == BUFFER_SIZE); // do nothing  
    buffer [in] = nextProduced;  
    in = ( in + 1 ) % BUFFER SIZE;  
    count++; //var partilhada  
}
```

PROCESSO CONSUMIDOR

```
while (true) {  
    while (count == 0)  
        ; // do nothing  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    count--; //var partilhada  
    /* consume the item in nextConsumed */  
}
```



consome

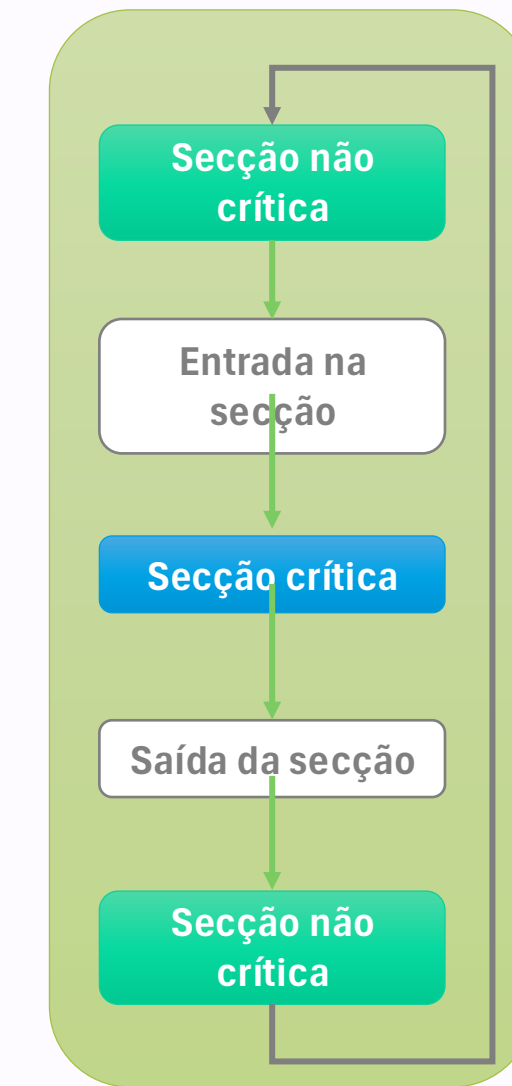
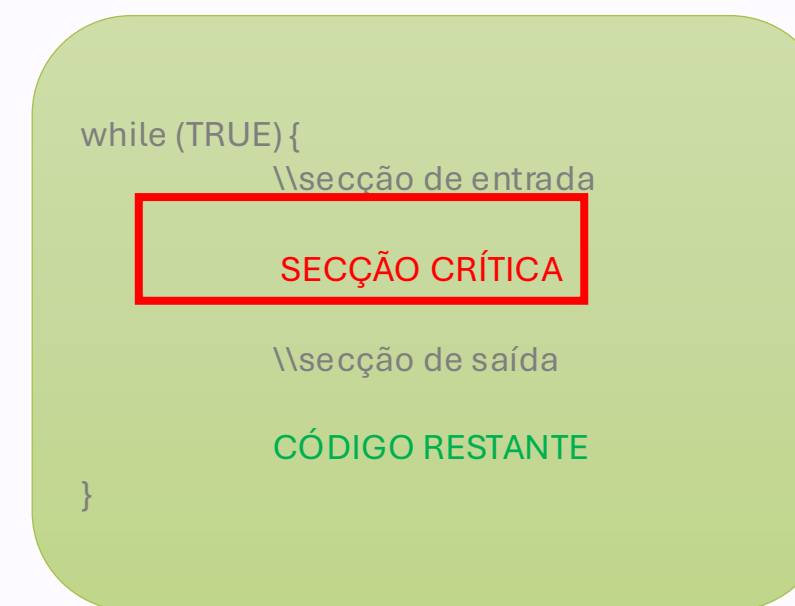
Buffer
(ou qualquer outra coisa partilhada)

RACE CONDITION

- Definição:
 - Resultado depende da ordem de execução
 - Processos concorrentes acedem dados partilhados
- Exemplo: count++ e count--
 - count++ → register1 = count; register1++; count = register1
 - count-- → register2 = count; register2--; count = register2
- Intercalação problemática:
 - T0: P1 load count (5) → reg1
 - T1: P2 load count (5) → reg2
 - T2: P1 inc reg1 (6)
 - T3: P2 dec reg2 (4)
 - T4: P1 store reg1 (6) → count
 - T5: P2 store reg2 (4) → count
 - Resultado: count = 4 (esperado: 5)
- Causa: Operações não atómicas

SECÇÃO CRÍTICA

- Definição:
 - Código que acede recursos partilhados
 - Apenas um processo por vez
- Estrutura:
 - Secção de entrada (request permission)
 - SECÇÃO CRÍTICA
 - Secção de saída (release permission)
 - Código restante
- Requisitos para solução:
 - 1. Exclusão mútua: um processo de cada vez
 - 2. Progresso: decisão não pode ser adiada indefinidamente
 - 3. Espera limitada: limite de vezes que outros entram
- Complexidade:
 - Sistemas multicore
 - Múltiplas threads
 - Preemption



MUTEX LOCK

- Conceito:
 - Mutual Exclusion Lock
 - Solução simples para secção crítica
- Operações:
 - acquire() - entrar na secção crítica
 - release() - sair da secção crítica
- Uso:
 - acquire()
 - SECÇÃO CRÍTICA
 - release()
- Implementação:
 - Variável booleana available
 - acquire: while (!available); available = false;
 - release: available = true;
- Problema:
 - Busy waiting (spinlock)
 - Desperdício de CPU
- Vantagem: Simples de implementar

SEMÁFOROS

- Definição:
 - Variável inteira com operações atômicas
- Operações:
 - `wait(S)` - decrementa, espera se $S \leq 0$
 - `signal(S)` - incrementa
- Tipos:
 - Binário (mutex): valores 0 ou 1
 - Contagem: valores 0 ou positivos
- Semáforo binário:
 - Semaphore mutex = 1;
 - `wait(mutex); CRITICAL; signal(mutex);`
- Semáforo de contagem:
 - Controla acesso a N instâncias
 - Semaphore S = 5; (5 recursos)
- Vantagens:
 - Mais flexível que mutex
 - Sincronização entre processos
- Problema: Ainda pode ter busy waiting

SOLUÇÃO PRODUTOR-CONSUMIDOR

- Com semáforos:
- Semaphore mutex = 1; // exclusão mútua
- Semaphore empty = N; // posições livres
- Semaphore full = 0; // posições ocupadas
- Produtor:
 - wait(empty); // espera espaço
 - wait(mutex); // entra secção crítica
 - adiciona item ao buffer
 - signal(mutex);
 - signal(full); // sinaliza item disponível
- Consumidor:
 - wait(full); // espera item
 - wait(mutex);
 - remove item do buffer
 - signal(mutex);
 - signal(empty); // sinaliza espaço livre

PROBLEMA LEITORES-ESCRITORES

- Cenário:
 - Base de dados partilhada
 - Leitores: apenas lêem
 - Escritores: lêem e escrevem
- Regras:
 - Múltiplos leitores simultâneos OK
 - Apenas um escritor por vez
 - Escritor exclui todos os leitores
- Solução com semáforos:
 - Semaphore mutex = 1; (protege readcount)
 - Semaphore wrt = 1; (exclusão escritor)
 - int readcount = 0;
- Leitor:
 - Incrementa readcount
 - Primeiro leitor bloqueia escritores
 - Último leitor liberta escritores
- Problema: Escritor pode ter starvation

DEADLOCK

- Definição:
 - Processos esperam indefinidamente
 - Evento só causado por processo em espera
- Exemplo:
 - P1: wait(S); wait(Q); ... signal(S); signal(Q);
 - P2: wait(Q); wait(S); ... signal(Q); signal(S);
 - Se P1 adquiere S e P2 adquiere Q → deadlock
- Condições necessárias (4):
 - 1. Exclusão mútua
 - 2. Hold and wait
 - 3. Sem preemption
 - 4. Espera circular
- Prevenção:
 - Ordem consistente de aquisição
 - Timeout
 - Detecção e recovery
- Detecção: Resource allocation graph

STARVATION

- Definição:
 - Processo nunca sai da fila de espera
 - Bloqueio indefinido
- Causas:
 - Fila LIFO (último a entrar, primeiro a sair)
 - Prioridades fixas
 - Scheduling injusto
- Exemplo:
 - Problema leitores-escritores
 - Escritor pode nunca executar se leitores contínuos
- Soluções:
 - Fila FIFO
 - Aging (aumentar prioridade com tempo)
 - Fair scheduling
 - Bounded waiting
- Diferença de deadlock:
 - Deadlock: ninguém progride
 - Starvation: alguns progridem, outros não

LOCK-FREE ALGORITHMS (2024-2026)

- Conceito:
 - Sincronização sem locks
 - Operações atômicas (CAS, fetch-and-add)
- Vantagens:
 - Sem deadlock
 - Melhor escalabilidade
 - Progress guarantee
- Resultados 2024-2026:
 - CMP queue: 1.72-4× vs state-of-art
 - Escalabilidade: centenas de threads
 - FIFO garantido + bounded reclamation
- Progress guarantees:
 - Wait-free: todos progridem
 - Lock-free: algum progride
 - Obstruction-free: progride se sozinho
- Desafios:
 - Complexidade de implementação
 - Memory reclamation (hazard pointers)
- Trade-off: Complexidade vs performance

TRANSACTIONAL MEMORY (2024-2026)

- Conceito:
 - Operações atômicas em blocos de código
 - Semelhante a transações de BD
- Sintaxe:
 - `atomic { // código transacional }`
- Tipos:
 - HTM (Hardware): Intel TSX
 - STM (Software): bibliotecas
 - HyTM (Hybrid): combina HTM + STM
- Resultados 2024-2026:
 - LockkillerTM: 1.86× vs HTM, 1.57× vs state-of-art
 - Máximo: 7.79× em cache pequeno
 - CHATS: 22% redução tempo execução
 - Persistent HyTM: ganhos em read-dominant
- Vantagens: Composabilidade, sem deadlocks
- Desvantagens: Capacity limits, aborts

NUMA-AWARE SYNCHRONIZATION (2024-2026)

- NUMA (Non-Uniform Memory Access):
 - Memória tem latências diferentes
 - Acesso local mais rápido
- Desafios:
 - Lock contention entre NUMA nodes
 - Cache coherence overhead
- Soluções modernas:
 - SCNA lock: space-efficient NUMA-aware
 - Organiza threads por node affinity
 - Adapta a contenção
 - CFL (Completely Fair Locking):
 - Fairness + NUMA topology
 - Melhor throughput
- Reciprocating Locks:
 - Low-latency uncontended
 - Menos coherence traffic
- Aplicações: Servidores multi-socket, HPC

GPU SYNCHRONIZATION (2024-2026)

- Desafios:
 - Milhares de threads
 - Warp-level synchronization
 - Memory consistency
- Primitivas:
 - `__syncthreads()` - block-level
 - Atomic operations
 - Memory fences
- Resultados 2024-2026:
 - Atomic Cache: 2.59× speedup, 1.48× IPC
 - >60% energy savings
 - 9.42% area overhead
 - OFG-STM: obstruction-free STM para GPU
 - LTPG: transaction processing em GPU
- In-cache atomics:
 - Reduz traffic para memória
 - Relaxed consistency para updates comutativos
- Aplicações: ML, graph processing, simulation

MANY-CORE SYNCHRONIZATION (2024-2026)

- Desafios:
 - Centenas/milhares de cores
 - Scalability crítica
 - Energy efficiency
- Polling-free synchronization:
 - LRSCwait/Colibri (256-core RISC-V):
 - 6.5× throughput
 - 7.1× energy efficiency
 - ~6% area overhead
 - Elimina busy polling
- MV-RLU (read-heavy):
 - 5.04× concurrency vs RCU (98% reads)
 - MVCC/transactional style
- Best practices:
 - Evitar polling quando possível
 - Sleep/queue em vez de spin
 - NUMA-aware placement

READER-WRITER LOCKS MODERNOS

- Problema clássico:
 - Múltiplos leitores, um escritor
- Variantes:
 - Reader-preference: leitores prioritários
 - Writer-preference: escritores prioritários
 - Fair: ordem de chegada
- Implementações modernas:
 - RCU (Read-Copy-Update): Linux kernel
 - Leitores sem locks
 - Escritores copiam e atualizam
 - SeqLock: otimizado para reads
 - Branchless RW locks
- Resultados 2024-2026:
 - MV-RLU: 5× vs RCU em 98% reads
- Trade-offs:
 - Performance vs fairness
 - Simplicidade vs overhead
- Escolha depende do workload

PRIORITY INVERSION

- Problema:
 - Tarefa baixa prioridade bloqueia alta prioridade
 - Tarefa média prioridade executa
- Exemplo:
 - T1 (alta): espera lock de T3
 - T2 (média): preempts T3
 - T1 bloqueada por T2 indiretamente
- Consequências:
 - Deadlines perdidos em RTOS
 - Caso famoso: Mars Pathfinder (1997)
- Soluções:
 - Priority Inheritance Protocol (PIP):
 - T3 herda prioridade de T1
 - Priority Ceiling Protocol (PCP):
 - Lock tem prioridade máxima
- Aplicações:
 - Sistemas de tempo real
 - Automotive, aviação, robótica

OPERAÇÕES ATÓMICAS

- Conceito:
 - Operação indivisível
 - Executa completamente sem interrupção
- Primitivas comuns:
 - Compare-And-Swap (CAS)
 - Fetch-And-Add
 - Test-And-Set
 - Load-Link/Store-Conditional (LL/SC)
- CAS pseudocódigo:

```
bool CAS(addr, expected, new) {  
    if (*addr == expected) {  
        *addr = new; return true;  
    } return false;  
}
```
- Memory ordering (C++11):
 - memory_order_relaxed
 - memory_order_acquire/release
 - memory_order_seq_cst
- Uso: Base de lock-free algorithms

FUTEX (FAST USERSPACE MUTEX)

- Conceito:
 - Mutex híbrido user/kernel space
 - Fast path: user space (atomic)
 - Slow path: kernel (syscall)
- Vantagens:
 - Baixa latência caso sem contenção
 - Kernel só quando necessário
- Operações:
 - `futex_wait(addr, val)`
 - `futex_wake(addr, n)`
- Uso:
 - Implementação de `pthread_mutex`
 - Condição variables
 - Semáforos POSIX
- Performance:
 - Uncontended: ~10-20ns
 - Contended: syscall overhead
- Estado 2024-2026:
 - Base de sincronização em Linux
 - Otimizações contínuas

C++20 SYNCHRONIZATION

- Novidades:
 - `atomic_ref<T>` - referência atômica
 - `std::barrier` - sincronização de fases
 - `std::latch` - contador único
 - `std::counting_semaphore`
 - `std::binary_semaphore`
- `atomic_ref`:
 - Atomicidade para variáveis não-atômicas
 - `atomic_ref<int> ar(x); ar++;`
- `barrier`:
 - Sincroniza grupo de threads
 - Reutilizável (múltiplas fases)
- `latch`:
 - Count-down, uso único
 - Espera até contador = 0
- Vantagens:
 - Standardização
 - Portabilidade
 - Type-safe

RUST E SINCRONIZAÇÃO

- Ownership model:
 - Previne data races em compile-time
 - Sem garbage collector
- Send e Sync traits:
 - Send: pode transferir entre threads
 - Sync: pode partilhar entre threads
- Primitivas:
 - Mutex<T> - exclusão mútua
 - RwLock<T> - reader-writer
 - Arc<T> - atomic reference counting
 - Atomic types (AtomicU32, etc.)
- Lock guards:
 - RAII - unlock automático
 - Previne esquecimento de unlock
- Vantagens:
 - Segurança em compile-time
 - Sem runtime overhead
- Limitações: Curva de aprendizagem

VERIFICAÇÃO FORMAL

- Objetivo:
 - Provar correção de algoritmos
 - Garantir propriedades (exclusão mútua, progress)
- Técnicas:
 - Model checking (SPIN, TLA+)
 - Theorem proving (Coq, Isabelle)
 - Static analysis
- Propriedades verificadas:
 - Safety: nada de mau acontece
 - Liveness: algo bom eventualmente acontece
 - Deadlock-freedom
 - Starvation-freedom
- Exemplos:
 - seL4: kernel formalmente verificado
 - Verificação de lock-free algorithms
- Desafios:
 - Complexidade
 - Escalabilidade
- Estado 2024-2026: Ferramentas melhoradas

BEST PRACTICES

- Design:
 - Minimizar secções críticas
 - Evitar nested locks
 - Ordem consistente de aquisição
- Performance:
 - Considerar lock-free para alta contenção
 - Reader-writer locks para read-heavy
 - Fine-grained locking quando apropriado
- Correção:
 - Usar ferramentas (ThreadSanitizer)
 - Testes de stress
 - Code review focado em concorrência
- Moderno:
 - Preferir linguagens type-safe (Rust)
 - Usar bibliotecas testadas
 - Considerar transactional memory
- Documentação:
 - Documentar invariantes
 - Ordem de locks

TENDÊNCIAS FUTURAS

- Lock-free dominante:
 - Adoção crescente
 - Bibliotecas maduras
- Transactional memory:
 - HTM melhorado
 - Persistent TM
- Many-core:
 - Sincronização escalável crítica
 - NUMA-aware essential
- GPU:
 - In-cache atomics
 - Relaxed consistency
- Language-level:
 - Rust model influencia outras linguagens
 - Type-safe concurrency
- Formal verification:
 - Ferramentas acessíveis
 - Verificação automática
- Hardware: Novos primitives

MÉTRICAS 2024-2026

- Lock-Free:
 - CMP queue: 1.72-4× vs state-of-art
- Transactional Memory:
 - LockillerTM: 1.86× (avg), 7.79× (max)
 - CHATS: 22% redução tempo
- NUMA-Aware:
 - SCNA lock: outperforms tradicionais
 - CFL: fairness + throughput
- GPU:
 - Atomic Cache: 2.59× speedup, >60% energy
- Many-Core:
 - LRSCwait: 6.5× throughput, 7.1× energy
 - MV-RLU: 5.04× vs RCU (98% reads)
- Geral:
 - Lock-free: 2-7× em alta contenção
 - NUMA: ~2× improvement

CONCLUSÕES

- Sincronização essencial em sistemas concorrentes
- Problema secção crítica: exclusão mútua
- Primitivas clássicas: mutex, semáforos
- Problemas: deadlock, starvation, priority inversion
- Modernos (2024-2026):
 - Lock-free: 1.72-4× speedup
 - Transactional memory: 1.86-7.79× gains
 - NUMA-aware: ~2× improvement
 - GPU: 2.59× speedup, >60% energy
 - Many-core: 6.5× throughput, 7.1× energy
- Trade-offs: simplicidade, performance, correção
- Futuro: lock-free, TM, language-level safety
- Best practice: Ferramentas + design cuidadoso

QUESTÕES?